実装計画: フォトアルバムオーガナイザー

プロジェクト: photo-album-organizer | 調査: 2026-01-16 | 対象: spec.md, requirements.md

概要

フォトアルバム・ボックアップ、以 ローカルファイルシステムを対象にした高速な重複検出とボックアップ。ハッシュベースの重複検出、ドライランモード、進捗表示、エラーリカバリを含む。Python 3.12+ / Typer CLI フレームワークを使用。

技術コンテキスト

言語/バージョン: JavaScript (ES6+) + HTML5 + CSS3

主要依存関係:
- Vue 3.5.13 (リアクティブ管理ライブラリ)
- vuje 1.0.0 (SQLite/WebAssembly 配信)
- Vitest 1.0.0 (テスト/カバレッジ)
- Serve 14.0.1 (ローカルサーバ)

ストレージ:
- ローカルストレージ (metadata, Obsidian)
- indexedDB容量 5GB, 2GB 推奨(Image 未使用) (SqueakCLIインポ)
- File API (ユーザ選択ファイルのみ読み込み)

テスト: Vitest (ユニット+統合) (コンポーネントテスト対応)

対象プラットフォーム:
- Chrome, Firefox, Safari, Edge (最新2版)
- デスクトップブラウザ のみ(モバイル

パフォーマンス目標:
- UI 応答時間 <1秒 (クリック/ドラッグ 可視反応、クリック表示)
- メモリ使用量 <200MB
- バンドルサイズ <100KB

制約:
- GitHub Pages 静的ホスティング (サーバ/コンポーネント)
- 外部 API 使用なし
- indexedDB 容量制限 (通常 5-10MB、ユーザ設定で拡張可能)

スケールスコープ:
- 初期対応: 1コンポ/100k+ 写真 x フォルダ
- 拡張予定: 2コンポ/10万写真 (将来)

憲法遵守チェック

ゲート: Phase 0 リサーチ前に合格すること。Phase 1 設計後に再評価します。

原則 I: データ整合性
- [x] データ損失ゼロ — コピーのみ (ソース削除なし)
- [x] ドライランモード — 実行前に何が起こるかを表示
- [x] 冪等性 — 繰り返し実行 = 同一結果
- [x] 進捗表示 — 処理中の状態を常に表示

原則 II: テストファーストの開発
- [x] TDD サイクル — テスト → 承認 → 失敗 → 実装
- [x] 統合テスト — ファイルシステム操作、エンドツーエンドフロー
- [x] ユニットテスト — ハッシュ計算、重複検出ロジック

原則 III: シンプルさ優先
- [x] プロジェクト数 — 1 (単一コンポーネント)
- [x] 抽象化 — 最小限 (ファイルシステムのみ)
- [x] フレームワーク — Typer のみ、過剰な機能なし

原則 IV: 可観測性
- [x] 構造化ログ — 全操作をタイムスタンプ付きで記録
- [x] エラーコンテキスト — 失敗時に完全なパスを含める
- [x] 進捗インジケータ — 長時間操作のリアルタイム更新

原則 V: セキュリティ優先
- [x] パスインジェクション — ユーザ入力のパスを検証
- [x] シンボリックリンク — デフォルトでフォローしない
- [x] 権限エラー — 優雅に処理、クラッシュしない

ゲート結果: 合格 ✅ (すべての原則が要件を満たす)

プロジェクト構造

ドキュメント

```
specs/005-photo-album-organizer/
├── spec.md              # ユーザーストーリー・要件
├── requirements.md      # 技術要件・実装ガイド
├── plan.md              # 実装計画 (このファイル)
├── research.md          # Phase 0 調査ドキュメント
├── data-model.md        # Phase 1 データモデル定義
├── quickstart.md        # Phase 1 クイックスタートガイド
└── contracts/           # Phase 1 インターフェース仕様
    ├── photo-storage-contract.md
    ├── album-storage-contract.md
    ├── order-persistence-contract.md
    └── tasks.md         # Phase 2 実装タスク (予定)
```

ソースコード

```
src/                     # メインコンテンツ・ロジックフォルダ
├── main.js              # アプリケーションエントリポイント (vuje初)
├── App.vue              # ルートコンポーネント
│                        # ・タブ管理・状態管理
│                        # ・ファイル選択処理
│                        # ・ドラッグ&ドロップ
│                        # ・進捗表示
│                        # ・エラー表示
│                        # ・ヒューリングツ
│                        # ・ドラッグ&ドロップ
├── components/
│   ├── MainPage.js      # メインページ (予定)
│   ├── ScanPage.js      # スキャンページ (予定)
│   ├── Filesize.js      # ファイルサイズ表示 (予定)
│   ├── FullscreenMode.js # フルスクリーンモード (予定)
├── utils/               # ユーティリティ
│   ├── DatabaseService.js # SQL.js DB 管理 (SAVE)
│   ├── AlbumService.js  # アルバムロジック・集計管理
│   ├── PhotoService.js  # 写真データ管理 (予定)
│   ├── MetadataCache.js # メタデータ管理 (予定)
├── utilis/              # ユーティリティ
│   ├── hashUtils.js     # SHA-256 ハッシュ・比較処理
│   ├── formatUtils.js   # フォーマット関数群 (formatBytes)
│   ├── logger.js        # ログシステム (save) (DEFLOGMSTORE)
│   ├── index.js
│   ├── package.lock     # 依存関係ロック
│   ├── components.css   # UI コンポーネント (button, card, modal)
│   ├── responsive.css   # レスポンシブ (breakpoints) (100px/768px/1024px)
tests/
├── unit/
│   ├── hashUtils.test.js # 日付処理テスト
│   ├── formatUtils.test.js # ファイルサイズ表示テスト
│   ├── AlbumService.test.js # アルバム・ロジックテスト
│   ├── logger.test.js   # ログシステムテスト
│   ├── integration/
│   ├── PhotoUpload.test.js # ファイル選択 → 保存
│   ├── AlbumMapping.test.js # タグ管理・グループ化
│   ├── Search.test.js   # 検索・フィルタリング
│   ├── Export.test.js   # ZIP生成・ダウンロード
│   ├── contract/
│   ├── DatabaseService.contract.test.js # DB CLI 検証
│   ├── AlbumService.contract.test.js # アルバム CLI 検証
│   ├── FileUtil.contract.test.js # ファイル操作 CLI 検証
public/
├── index.html           # GitHub Pages 用ランディング
├── favicon.ico
package.lock
package.json
vite.config.js
vitest.config.js
```

→ 単一プロジェクト (フロントエンド)

```
vite.config.js           # Vite 設定 (base: '/photo-album-organizer/')
package.lock             # 依存関係・スクリプト定義

→ 単一ポイントエントリ:

  - GitHub Pages で配 (photo-album-organizer/) パスを相対のため Vite の base 設定
  - アルバム (Album) と写真 (Photo) 分離 を管理
  - サイズ順で重複グループを検出し、ドライランモードを提供
  - sql.js 経 IndexedDB 永続化 (ページリロード後もデータ保持)
  - 手動バックアップ (エクスポート/インポート) 実装
```

複雑性トラッキング

違反なし

フォローアップ: 将来の Constitution Check 違反は、ここに追加してください。

違反	必要な理由	より単純な代替案が不適切な理由
(なし)		

Phase 0: 技術リサーチ (予定)

リサーチ対象:

1. **SQLite.js (sql.js) in indexedDB ストレージ**
   - 決定: sql.js - 完全なSQLite 機能 (複雑なクエリ対応可能)
   - 根拠: アルバム比較で必要 (メタデータ検索 SQL 評価中)
   - 検討: プロ中 (アルバム比較 プロ中)
   - 対象: アルバム・SQLite (複雑 SQL 可能対)

2. **Vue 最適化戦略**
   - Code Splitting: コンポーネント遅延ロード
   - Tree Shaking: 未使用コード削除
   - Minify: Terser でコード圧縮
   - 目標: バンドル < 100KB (現在: 62KB 可能対)

3. **大規模ファイル処理戦略**
   - 仮想スクロール: 1000+ 写真表示時のレンダリング
   - 遅延ロード: 必要なときだけ メタ情報読み込み
   - メモリ管理: ファイル数 プロ中 (メモリ使用量 プロ中)
   - 手動バッチ: 各プロ中 (バッチ処理 実装)

4. **ブラウザ互換性確認**
   - 対象: Chrome/Firefox/Safari/Edge 最新2版
   - File API: indexedDB 容量 5GB 推奨 (制限 5-10MB)
   - 手動バッチ: 各プロ中 (バッチ処理 実装)

出力: research.md (Phase 0)

最終更新: 初期作成 (残 未調査 未決定 未決定)

Phase 1: 設計 — コントラクト (予定)

1. データモデル定義 (data-model.md)

エンティティ

- **Photo**
  - id: UUID
  - file_name: string (max 255)
  - file_path: string (unique)
  - photo_date: ISO 8601 (YYYY-MM-DD)
  - photo_time: ISO 8601 (HH:MM:SS)
  - date_str: string (based-encoded image)
  - created_at: ISO 8601
  - checksum: string (MD5 of file content, for deduplication)
- **Album**
  - id: UUID
  - album_date: ISO 8601 (YYYY-MM-DD, unique)
  - display_order: number (for sorting)
  - created_at: ISO 8601
  - photo_count: number (denormalized for UI)
- **AlbumOrder** (or stored in Album.display_order)
  - album_id: UUID
  - display_order: number

インデックス

- photo(photo_date): 日付範囲検索
- photo(created_at): クリエイティブ順序
- album(display_order): 表示順序ソート
- album(album_date): 日付検索

制約

- album.album_date: UNIQUE
- photo.photo_date: NOT NULL
- foreign key: photo(album_id) → album(id)

2. コントローラ/ストア (contracts/)

- **photo-storage.contract.md**
  - getPhotoBy(id: Photo) → Promise<Photo>
  - getPhotosBy(filter: {date?, hasAlbum?}) → Promise<Photo[]>
  - savePhoto(data: Photo) → Promise<void>
  - updatePhotoOrder(photos: Photo[]) → Promise<void>
- **album-storage.contract.md**
  - createAlbum(date: string) → Promise<Album>
  - getAlbumBy(filter: {date?, id?}) → Promise<Album>
  - updateAlbumOrder(albums: Album[]) → Promise<void>
  - deleteAlbum(id: UUID) → Promise<void>
- **order-persistence.contract.md**
  - saveAlbumOrder(order: AlbumOrder[]) → Promise<void>
  - loadAlbumOrder() → Promise<AlbumOrder[]>

3. ディリクエスト (quickstart.md)

開発者向けクイックガイド

- 開発環境 (Node.js, npm)
- 1ポリクロン ローカルビルド
- 1ポリクロン ローカルサーバ 起動
- Vitest テスト (unit / contract)
- CLI ガイド (Typer CLI) (GitHub Pages)
- トリ方ルーティング

Phase 2: タスク分解 (予定)

/speckit.tasks コマンドで生成予定

tasks.md で詳細 (予定)

コーポ・コントローラ・UI: 並列実行 (Phase 1 後)

- Task 1.1: メタデータ解析ユニット (TDD)
- Task 1.2: DatabaseService.addPhoto() 実装
- Task 1.3: 重複検出ロジック (SHA-256 ハッシュ) 実装
- Task 1.4: ドラッグ/ドロップ (D&D API)
- Task 1.5: ファイルリスト (INSERT OR)

コーポ・コントローラ・UI: 逐次実行 (Phase 1 後)

- Task 2.1: Timeline コンポーネント実装
- Task 2.2: 重複グループ表示
- Task 2.3: 進捗インジケータ (SQL)
- Task 2.4: ドラッグ/ドロップ (PWA API)

コーポ・コントローラ・UI: 逐次 (タイ)最終 (Ok)

- Task 3.1: Fullscreen コンポーネント実装
- Task 3.2: ドラッグ/ドロップ実装
- Task 3.3: メタデータ解析 (EXIF)

実装状況 (現在)

現在 7 重目 ✅

- ✅ 日付アルブ (Hooks, sql.js, Vite 環境構築
- ✅ package.json: 依存関係定義 (完了)
- ✅ vite.config.js: GitHub Pages base path (/photo-album-organizer/) 設定
- ✅ index.html: ビルド (コピー) メインページ アルバム(2) ~ フルサイズ (Ok)
- ✅ main.js: アルバ(ビルド)アプリ ローカルロード
  - ✅ 初期化 (DB 接続)
  - ✅ メタ・ドラッグ・D&D
  - ✅ アルバム・ビルド・ロジック
  - ✅ アルバム・ビルド・ロジック
  - ✅ ビルド・ビルド・ビルド
- ✅ DatabaseService: SQLite CLI DB (OK)
  - ✅ initDatabase()
  - ✅ initialize(savedBuf)
  - ✅ addPhoto(), getPhotoBy(date), getAllAlbumDates()
  - ✅ getOrCreateAlbum(), saveDatabase()
- ✅ AlbumMapping: ビルド ロジック
  - ✅ groupPhotosBy()
  - ✅ getAlbums()
  - ✅ updateAlbumOrder()
- ✅ utilities: dateUtils, formatUtils, logger
- ✅ styles: main.css, components.css, responsive.css
- ✅ 最適化 (Phase 2): 仮想スクロール, 遅延ロード, メモリ管理
- ✅ 手動バッチ: 各プロ中 (バッチ処理 実装)
- ✅ npm install: ローカル パッケージ インストール完了
- ✅ npm run build: ビルド成功 (62KB)
- ✅ Git commits: メイン中 (完了)

次の重要 ⏳

- ⏳ research.md: Phase 0 調査ドキュメント作成
- ⏳ data-model.md: データ モデル定義
- ⏳ contracts/: コントローラ・ストア
- ⏳ quickstart.md: 開発者ガイド
- ⏳ タスク分解:
  - ⏳ コンポ・ビルド (Dockerfile, Kubernetes, etc.)
  - 遅延タスク: Dockerfile, Kubernetes, Helm
  - ⏳ コントローラ: DatabaseService, AlbumService
- ⏳ **tasks.md**: Phase 2 タスク分解 (/speckit.tasks コマンド)

ビルド・デプロイメント

開発

```
npm run dev              # localhost:5173 起動サーバ
npm run build            # dist/ ビルド
npm run test:ui          # Vitest UI 起動
```

本番

```
npm run build            # dist/ ビルド (62KB)
npm run preview          # ビルド済みプレビュー
rsync -r dist/ user@host:/var/www/photo/
```

パフォーマンス検証

```
# Lighthouse スコア測定
npm run build
npm run preview
# → ブラウザで http://localhost:4173 を Lighthouse で検査

# メモリ プロファイリング
# Chrome DevTools → Memory → Heap Snapshot
```

次のステップ

1. **Phase 0 実行** (/speckit.plan): research.md 生成
2. **Phase 1 実行**: data-model.md, contracts/ 生成
3. **agent-context 更新**: copilot-instructions.md 更新
4. **Phase 2 実行** (/speckit.tasks): tasks.md 生成
5. **タスク実装**: TDD に従い、各タスクごとにテスト
6. **コーポ実装**: 各タスクのテスト合格 → 実装